SYSTEM DESIGN

Message Queues Visual Guide

Decouple producers from consumers: buffers, ordering, and delivery guarantees.

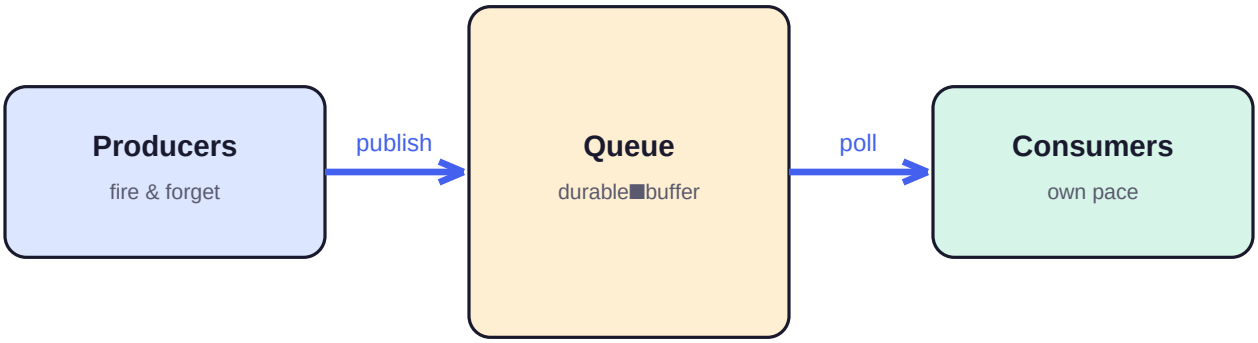
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Producer / Consumer Decoupling

A durable buffer between 'work created' and 'work done'.

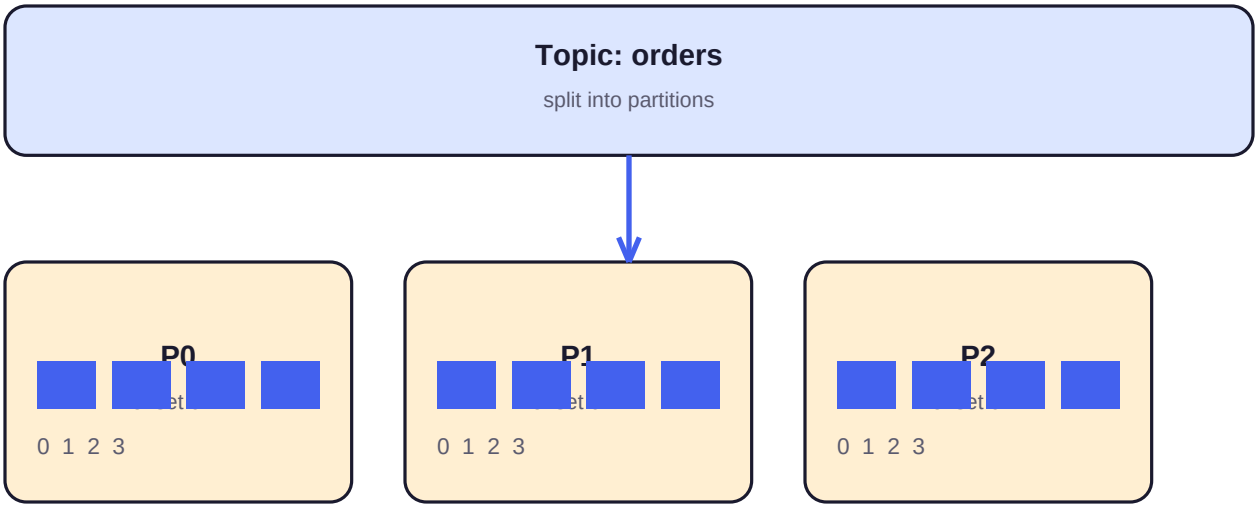


- Absorbs bursts: producers spike at 10x, consumers drain steadily -- the queue is the shock absorber.
- Decouples in time: consumer can be down for hours; messages wait. Decouples in space: neither side knows the other.
- Backpressure done right: slow consumers show as queue DEPTH (a metric), not as timeouts upstream.

STEP 2/6

Kafka: Partitions and Offsets

The log is the queue; partitions are the parallelism.



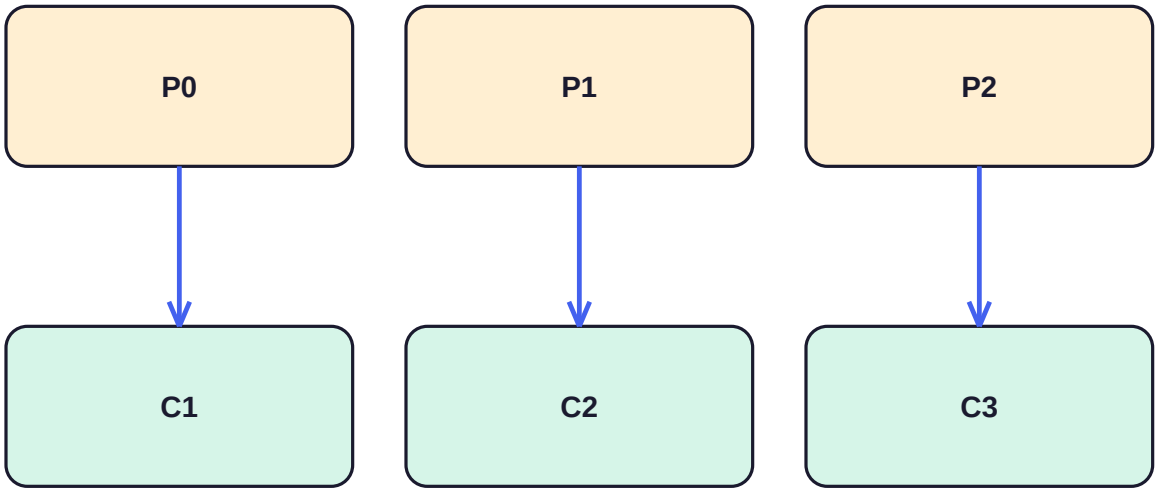
Ordering guaranteed WITHIN a partition only. Same key -> same partition -> ordered.

Consumers track their own offset per partition: replay by rewinding. The broker never deletes what you have not read.

STEP 3/6

Consumer Groups Scale the Work

One partition -> one consumer in a group: add consumers, add throughput.



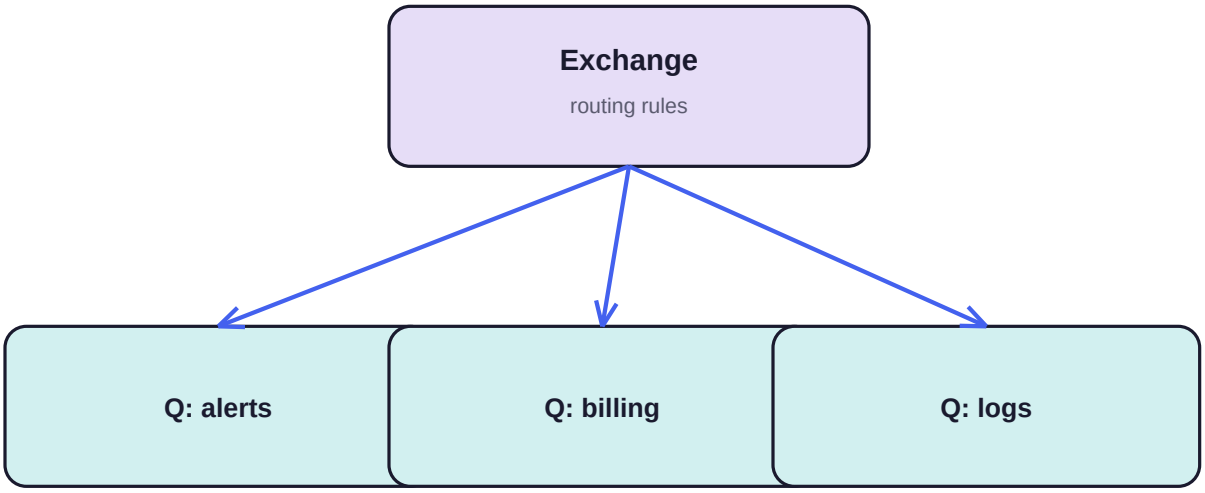
Ceiling: a group cannot out-consume its partition count. Need more parallelism? Add partitions (careful: key ordering).

- Rebalance: when a consumer joins/dies, partitions are reassigned -- expect a brief pause.
- Two groups on one topic = the same data feeding two pipelines independently.

STEP 4/6

RabbitMQ: Exchanges and Routing

The exchange decides WHO gets the message; queues just hold them.



- Fanout: broadcast to every bound queue. Direct: exact routing-key match.
- Topic: pattern match (orders.* vs orders.eu.#). Headers: match on message attributes.
- Contrast Kafka: RabbitMQ routes smartly to queues; Kafka keeps one dumb log and lets consumers choose.

STEP 5/6

Delivery Semantics

What 'guaranteed delivery' actually promises.

At-most-once

fire and forget -- may LOSE messages (metrics, logs)

At-least-once

ack required, retry on timeout -- may DUPLICATE (the default)

Exactly-once

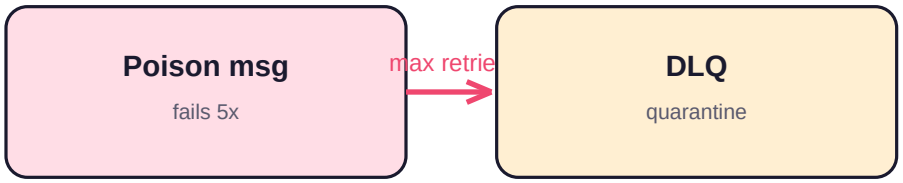
effect applied once -- needs idempotent consumers + transactions

Practical exactly-once = at-least-once delivery + idempotent receiver. See the Idempotency guide.

STEP 6/6

Dead Letters and Poison Messages

Some messages will never succeed. Plan their funeral.



- Retry with backoff first; after N failures route to the Dead-Letter Queue -- never block the main queue.
- DLQ is an inbox for humans: inspect, fix, replay. Alert on DLQ depth, not just consumer lag.
- Poison messages (malformed, giant) deserve a schema registry + size limits so they never enter.

STEP ✓/✓

Key Takeaways

Message Queues — Remember

- Queues decouple producers and consumers in time and space; depth is your backpressure metric.
- Kafka: partitioned append-only logs, per-partition ordering, consumer offsets enable replay.
- Consumer groups parallelize to the partition count; rebalances pause consumption briefly.
- RabbitMQ exchanges route (fanout/direct/topic); Kafka keeps one log and lets consumers filter.
- Default to at-least-once + idempotent consumers; quarantine poison messages in a DLQ.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev